



Tecnológico de Monterrey

Reflexión individual – Eficiencia de los algoritmos de búsqueda y ordenamiento

Franco De Escondrillas Vázquez | A01739410

Ingeniería en Tecnologías Computacionales

Dr. Daniel Pérez Rojas

TC1031.601

21 de septiembre, 2025

Introducción

El conocimiento de distintos tipos de algoritmos, tanto de búsqueda como de ordenamiento, resulta esencial para el desarrollo de programas ágiles capaces de operar en el menor tiempo posible. El motivo por el cual esto resulta importante es que el desempeño de cada algoritmo podrá variar dependiendo del contexto en el que se use, por lo que es responsabilidad del programador saber qué método resulta más adecuado.

En esta unidad de formación se revisaron dos tipos de búsqueda y seis de ordenamiento. En el primer caso, se revisó con detenimiento el funcionamiento y complejidad de tiempo de la búsqueda secuencial y la binaria, las cuales se explicarán en una sección posterior. Por otro lado, los algoritmos de ordenamiento vistos fueron *swapsort*, *insertionsort*, *selectionsort*, *bubblesort*, *quicksort* y *mergesort*.

Algoritmos de búsqueda

Búsqueda secuencial

Este tipo de búsqueda es el más básico y común. La gran ventaja de este es que se puede utilizar en cualquier contexto, independientemente de que los datos se encuentren ordenados, desordenados o incluso se manejen tipos de datos diferentes. A pesar de su versatilidad, la complejidad de tiempo es bastante elevada; en el mejor de los casos es necesario recorrer todo el arreglo para encontrar el elemento que se busca. Por ello, su complejidad temporal es lineal $O(n)$.

Búsqueda secuencial $O(n)$

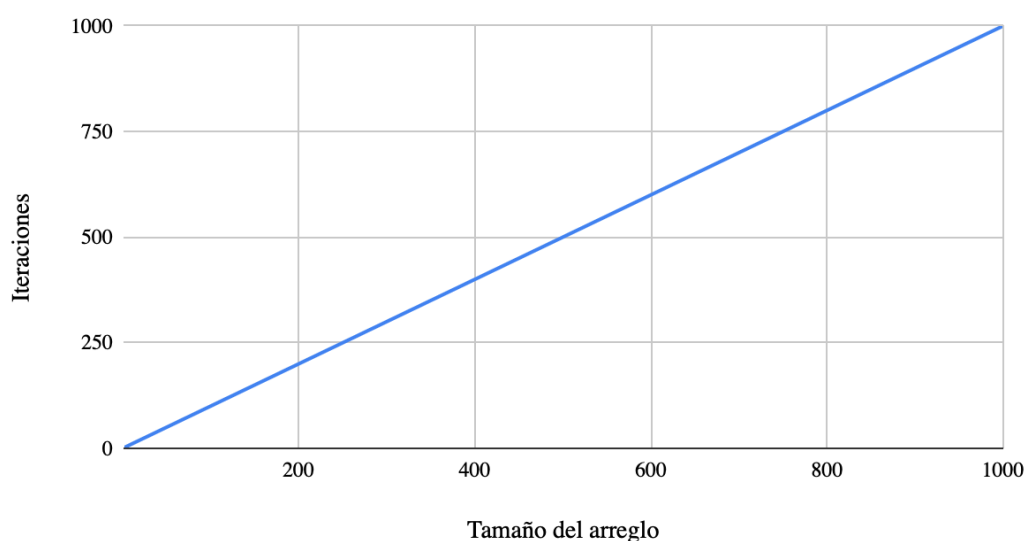


Figura 1. Complejidad búsqueda secuencial

Búsqueda binaria

Este segundo tipo de búsqueda resulta mucho más eficiente. La forma en que funciona es partiéndolo el conjunto de datos a la mitad, evaluando el elemento medio y descartando la mitad que contenga elementos fuera del rango de aquello que se busca, por lo que su complejidad es de $O(\log_2 n)$. Sin embargo, para que esto funcione, el conjunto de datos debe encontrarse ordenado.

Búsqueda binaria $O(\log_2 n)$

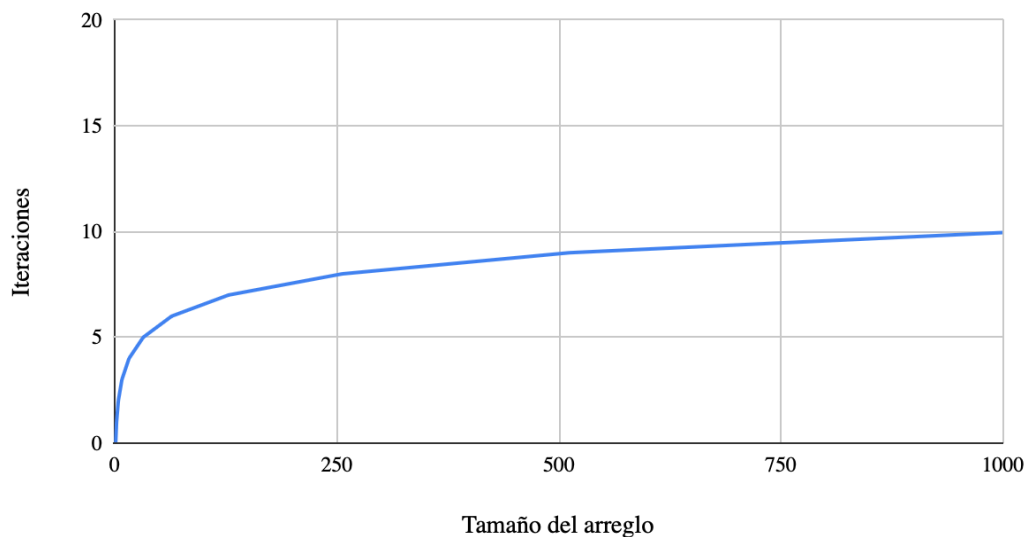


Figura 2. Complejidad temporal de la búsqueda binaria

Algoritmos ordenamiento

Los casos revisados en clase se pueden dividir en dos grupos: aquellos que requieren de una comparación directa de todos los datos contra todos, y aquellos que utilizan estrategias de separación de datos para optimizar el número de comparaciones. En el primer grupo se encuentran *swapsort*, *insertionsort*, *selectionsort* y *bubblesort*, que aunque tienen ligeras diferencias entre sí, comparten una complejidad temporal de $O(n^2)$.

Por otro lado, los algoritmos *quicksort* y *mergesort* comparten una técnica en la que el conjunto de datos se comienza a separar para ordenar grupos de datos más pequeños y reducir la complejidad de tiempo. Dentro de estos dos, el primero de ellos tiende a una complejidad temporal de $O(n \log_2 n)$ en el mejor de los casos, pero un $O(n^2)$ en el peor. Por otro lado, el segundo ejemplo funciona siempre con una complejidad de $O(n \log_2 n)$. Es importante mencionar que la forma en que ambos algoritmos se implementan es naturalmente recursiva, lo que difiere de los anteriores que son naturalmente iterativos.

Tabla 1. Complejidad temporal de los algoritmos de búsqueda.

Algoritmo	Complejidad temporal		
	Mejor caso	Caso promedio	Peor caso
Selectionsort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertionsort	$O(n)$	$O(n^2)$	$O(n^2)$
Bubblesort	$O(n)$	$O(n^2)$	$O(n^2)$
Swapsort	$O(n)$	$O(n^2)$	$O(n^2)$
Quicksort	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n^2)$
Mergesort	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$

Conexión con la situación problema

El vínculo entre los algoritmos de búsqueda y ordenamiento y la ciberseguridad puede no ser evidente a primera vista, pero es crucial. En la detección de accesos maliciosos, ataques cibernéticos y botnets, el problema central radica en manejar cantidades masivas de datos, donde es necesario hacer análisis constantemente para detectar accesos maliciosos o nodos comprometidos, por ejemplo.

La importancia de conocer los diferentes algoritmos es que, aunque en el manejo general de logs es lógico emplear una combinación entre *mergesort* y búsqueda binaria, en un contexto en el cual los logs se produzcan en tiempo real, entonces podría ser mejor usar una búsqueda secuencial y un *insertionsort* para ingresar los datos nuevos, una vez se han ordenado los registros anteriores.